

# Reviewer Pack — Minimal Tropicalized Local Cohomology Order and Resolution P...

Entry dcf4f91573a0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 03:10:06 UTC

## Minimal Tropicalized Local Cohomology Order and Resolution Proof Width Correlation

**Entry ID:** dcf4f91573a0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 03:10:06 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry (Local Cohomology) **Field B** (complexity object): Resolution Proof Complexity

#### Statement:

For every Boolean formula  $\varphi$  with  $n$  variables, the minimal tropicalized local cohomology order ( $\text{mloc}(\varphi)$ ) of its associated circuit is linearly correlated with its resolution proof width  $w(\varphi)$ , such that  $\text{mloc}(\varphi) = \Theta(w(\varphi))$ .

#### Rationale (proposer's reasoning):

Local cohomology theory provides a geometric interpretation of algebraic varieties, and tropicalizing this theory may reveal new structural insights into the complexity of circuits. A correlation between tropicalized local cohomology order and resolution proof width could indicate a deeper connection between geometry and computational complexity.

**Taxonomy category:** TROPICAL\_FOURIER\_ANALYSIS (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** c36a1cb0caefd5de

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between  $\text{mloc}(\varphi)$  and  $w(\varphi)$  across all generated Boolean formulas is greater than or equal to 0.8, with no seed producing a correlation below this threshold. The conjecture is falsified if any seed produces a correlation coefficient less than 0.8.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 1.00       | UNCERTAIN        | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=240.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_circuit(formula):
        n = len(formula)
        circuit = []
        for i in range(n):
            if formula[i] == 0:
                circuit.append((i,))
            else:
                circuit.append((i, i+1))
        return circuit

    def compute_tropicalized_local_cohomology_order(circuit):
        n = len(circuit)
        max_rank = 0
        for node in range(n):
            rank = 0
            for edge in circuit:
                if node in edge:
                    rank += 1
            max_rank = max(max_rank, rank)
        return max_rank

    def compute_resolution_proof_width(circuit):
        n = len(circuit)
        cnf = []
        for i in range(n):
            cnf.append([i])
        for edge in circuit:
            if len(edge) == 2:
```

```

        cnf.append([-edge[0], edge[1]])
    return solve_cnf(cnf, n)

def solve_cnf(cnf, n):
    def dpll(lits, cls):
        if not lits:
            return True
        lit = next((l for l in range(n) if l not in cls and -l not in cls), None)
        if lit is None:
            return False
        new_lits_true = [l for l in lits if l != lit]
        new_lits_false = [l for l in lits if l != -lit]
        return dpll(new_lits_true, cls | {lit}) or dpll(new_lits_false, cls | {-lit})

    return dpll(cnf, set())

n_values = [5, 10, 15, 20, 30, 40]
correlation_coefficient = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        formula = generate_boolean_formula(n)
        circuit = compute_circuit(formula)
        mloc = compute_tropicalized_local_cohomology_order(circuit)
        w = compute_resolution_proof_width(circuit)
        correlation_coefficient += (mloc - 10) * (w - 10)
        instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

correlation_coefficient /= instances_tested
return {
    "metric_name": "correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_va
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fractions}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no seeds tested")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out before producing data, which means it was unable to complete the analysis required to determine the Pearson correlation coefficient between  $mloc(\varphi)$  and  $w(\varphi)$ . As a result, we cannot confirm or refute the conjecture based on this test. | next: NONE

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15042        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9836         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9148         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 20793        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20205        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 43143        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 27666        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 30897        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 18567        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 195297 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/dcf4f91573a0.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dcf4f91573a0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/dcf4f91573a0.tar.gz (if generated)